

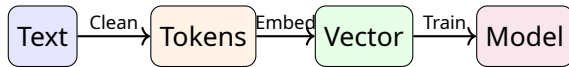
Natural Language Processing (NLP)

Preprocessing, Embeddings, Transformers & Hugging Face

1 Text Preprocessing

Libraries

```
import nltk
import spacy
nlp = spacy.load("en_core_web_sm")
```



Tokenization

Splitting raw text into smaller units called tokens (words, subwords, or characters).

```
text = "Hello world! It's NLP."

# NLTK (Rule-based)
tokens = nltk.word_tokenize(text)
# ['Hello', 'world', '!', 'It', "'s", ...]

# spaCy (Object-oriented)
doc = nlp(text)
tokens = [token.text for token in doc]
```

Cleaning & Normalization

Standardizing text by removing noise (stopwords, punctuation) and normalizing case.

```
# Lowercasing
tokens = [t.lower() for t in tokens]

# Removing Stopwords (NLTK)
from nltk.corpus import stopwords
stop = set(stopwords.words('english'))
clean = [w for w in tokens if w not in stop]

# Lemmatization (spaCy)
# Converts 'running' -> 'run', 'better' -> 'good'
lemmas = [token.lemma_ for token in doc]
```

2 Feature Extraction (Traditional)

Bag of Words (BoW)

Represents text as a fixed-length vector of word counts, ignoring grammar and order.

```
from sklearn.feature_extraction.text \
import CountVectorizer

corpus = ["I love NLP", "NLP is fun"]
cv = CountVectorizer()
X = cv.fit_transform(corpus)
# Output: Sparse matrix of counts
print(cv.get_feature_names_out())
# ['fun', 'is', 'love', 'nlp']
```

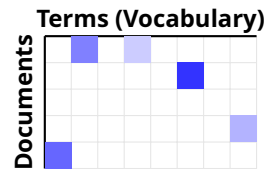
TF-IDF

Term Frequency - Inverse Document Frequency. Weighs words by their importance (rare words get higher weights).

$$w_{i,j} = tf_{i,j} \times \log\left(\frac{N}{df_i}\right)$$

```
from sklearn.feature_extraction.text \
import TfidfVectorizer

tfidf = TfidfVectorizer()
X = tfidf.fit_transform(corpus)
```



N-Grams

Capture context by grouping N consecutive words.

```
# Context window of 2 words (Bigrams)
cv = CountVectorizer(ngram_range=(2, 2))
# Features: ["I love", "love NLP", ...]
```

3 Word Embeddings

Dense vector representations where words with similar meanings are close in vector space.

Word2Vec (Gensim)

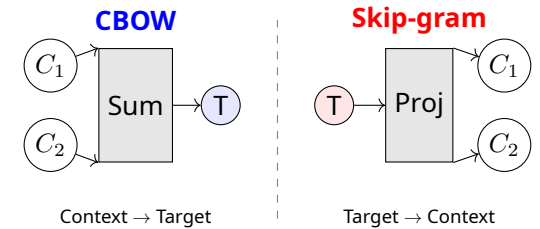
Learns embeddings by predicting a word from context (CBOW) or context from a word (Skip-gram).

```
from gensim.models import Word2Vec

sentences = [["cat", "sat"], ["dog", "barked"]]
# vector_size: dimension (e.g., 100, 300)
# window: context distance
# min_count: ignore rare words
model = Word2Vec(sentences, vector_size=100,
                  window=5, min_count=1)

# Get vector
vec = model.wv['cat']

# Similarity
sim = model.wv.most_similar('dog')
```



GloVe (Pre-trained)

Global Vectors. Embeddings derived from aggregate word-word co-occurrence statistics.

4 Transformers (Hugging Face)

State-of-the-art models (BERT, GPT) using Self-Attention to handle long dependencies.

Pipeline (High Level)

```
from transformers import pipeline

# Sentiment Analysis
clf = pipeline("sentiment-analysis")
res = clf("I love this library!")
# [{'label': 'POSITIVE', 'score': 0.99}]

# Translation
trans = pipeline("translation_en_to_fr")
print(trans("Hello world"))
```

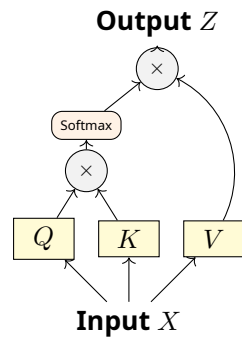
Tokenizer & Model (Low Level)

```
from transformers import AutoTokenizer, \
AutoModelForSequenceClassification

name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(name)
model = AutoModelForSequenceClassification \
    .from_pretrained(name)

# 1. Tokenize (Input -> IDs)
inputs = tokenizer("Hello world",
                  return_tensors="pt")

# 2. Forward Pass
outputs = model(**inputs)
logits = outputs.logits
```



Self-Attention

Mechanism allowing the model to weigh the importance of different words in the input sequence relative to each other.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

5 Common NLP Tasks

Named Entity Recognition (NER)

Classifies entities into categories like Person, Organization, Date.

```
ner = pipeline("ner")
```

```
ner("Apple bought a startup in NYC.")
# [Apple (ORG), NYC (LOC)]
```

Part-of-Speech (POS) Tagging

Assigns grammatical parts of speech (Noun, Verb, Adjective) to tokens.

```
doc = nlp("They run fast")
# run -> VERB, fast -> ADV
```

Text Summarization

```
summ = pipeline("summarization")
article = "... long text ..."
print(summ(article, max_length=50))
```

6 Evaluation Metrics

Classification

- **Accuracy, Precision, Recall, F1.**

Translation / Summarization

- **BLEU:** Precision-based. Checks n-gram overlap between candidate and reference.
- **ROUGE:** Recall-based. Common for summarization evaluation.

Language Modeling

- **Perplexity:** Measures how well a model predicts a sample. Lower is better.

$$PP(W) = \sqrt[N]{\frac{1}{P(w_1 \dots w_N)}}$$

7 Advanced Concepts

Fine-Tuning

Adapting a large pre-trained model (like BERT) to a specific downstream task/dataset.

```
from transformers import Trainer, TrainingArguments

args = TrainingArguments(output_dir="test_trainer")
trainer = Trainer(model=model, args=args,
                  train_dataset=train_dataset)
trainer.train()
```

Handling Long Text

- **Truncation:** Simply cutting off text after 512 tokens.
- **Sliding Window:** Processing chunks with overlap.
- **Longformer/BigBird:** Architectures designed for long sequences.